深度学习

坦克大战AI系统技术实现

尊敬的各位老师、同学们：

大家好！我将基于我们的项目技术文档，为大家详细介绍基于分层深度强化学习的坦克大战AI系统。

一句话概括

DQN是一种将**深度学习**与**Q-learning**相结合的强化学习算法，它让AI智能体能够直接从高维的原始输入（如图像）中学习如何做出最优决策。

第一部分：整体架构设计

各位好，先从**基础层**开始 —— 这是咱们 AI 系统的 “地基”，负责把游戏信息 “翻译成 AI 能懂的语言”。

左边这个 `ai_model.py` ，是 AI 的 “大脑结构图纸”：它里面有三个核心模块 —— 先通过卷积网络把游戏画面（84×84 的图）变成 256 个数字的 “画面特征”，再用全连接网络把游戏数据（比如血量、敌人位置）也变成 256 个数字的 “状态特征”，最后把这两组特征融合，输出 “该做什么动作” 的判断。

右边的 `ai_state_extractor.py` ，是 AI 的 “信息采集器”：它先把游戏的彩色大图（800×600）压缩成小的灰度图，再从游戏里扒出 85 个关键数据（比如玩家状态、敌人信息），把这些 “原始素材” 整理成标准格式，给大脑模块当 “食材”。

简单说，基础层就是 “把游戏信息加工成 AI 能理解的原料”，是整个系统的 “信息转换器”。

（切换到第二张 PPT：训练层核心文件功能）

有了基础层的 “原料”，接下来是**训练层**—— 这是让 AI “从新手变高手” 的环节。

第一个 `ai_training_env.py` ，是 AI 的 “训练模拟器”：它把游戏环境标准化，比如规定 “画面要多大、能做哪些动作”，还能调难度（比如敌人数量、基地血量），给 AI 搭一个 “虚拟训练场”。

第二个 `ai_train.py` ，是 AI 的 “底层动作训练”：它让 AI 在模拟器里反复玩（最多玩 10 万局），每一步都调整 AI 的决策逻辑，还会记录 “得分、失误” 这些数据，帮 AI 优化 “走、打、躲” 这些基础动作。

第三个 `ai_train_option.py`，是 AI 的 “高层策略训练”：它教 AI 做 “战术选择”—— 比如什么时候进攻、什么时候防守、什么时候绕后，把这些策略和底层动作结合，让 AI 不止会 “操作”，还会 “思考战术”。

训练层的目标很明确：让 AI 从 “乱按按键” 变成 “会打游戏的老手”。

（切换到第三张 PPT：应用层核心文件功能）

最后是**应用层**—— 这是把训练好的 AI “投入实战” 的环节。

左边的 `ai_agent.py`，是 AI 的 “完整身体”：它把前面所有模块都整合起来 —— 从 “看画面、读数据”，到 “选战术、做动作”，甚至还加了 “安全检查”（比如躲子弹）、“防卡死”（卡 bug 了自动动一动）这些实用功能，实现从 “感知” 到 “行动” 的全流程。

右边的 `test_ai.py`，是 AI 的 “功能验证与评估”：它会让 AI 打 100 多局游戏，然后打分 —— 比如胜率多少、平均活多久、能杀多少敌人，最后把这些数据记下来，验证 AI 实战到底好不好用、稳不稳定。

应用层就是把 “训练好的 AI” 变成 “能真正上场的选手”，同时验证它的实战能力。

收尾

总结一下：咱们的系统是 “三层递进” 的 —— 基础层做信息加工，训练层让 AI 学本领，应用层让 AI 打实战，三层配合起来，就能实现一个能自己玩游戏的智能体。

第二部分：核心技术组件详解

接下来咱们看 “双模态特征提取系统”—— 这是 AI “看懂游戏” 的核心能力，相当于给 AI 配了 “眼睛 + 数据雷达”。

先看左边的**视觉特征提取**：游戏原始画面是 800×600 的彩色图，我们会把它变成黑白图（灰度化）、缩小到 84×84（方便 AI 处理）、再把数值统一到 - 1 到 1 之间，最后输出一个标准的图像数组 —— 这一步是让 AI“看清楚画面里的场景、物体”。

中间的**数值特征提取**，是给 AI 装 “数据雷达”：它会从游戏里扒出 6 大类、85 个关键数据 —— 比如玩家在哪、血量多少，敌人的位置，子弹飞哪了，有什么道具，基地在哪等等。这些数据是 AI 判断 “局势” 的硬信息。

而右边的**双模态作用**，就是把 “看画面” 和 “读数据” 结合起来：画面帮 AI 感知 “空间信息”（比如敌人在画面左边还是右边），数据帮 AI 量化 “具体状态”（比如敌人血量还剩多少），两者一结合，为后续决策提供输入

（切换到第二张 PPT：DQN 网络架构）

接下来就是 AI 的 “大脑核心”——DQN 网络架构，它负责把 “输入” 变成 “决策”。

我分三部分讲：**第一部分，==CNN 视觉特征提取器==：这是处理 “画面原料” 的模块，用 3 层卷积网络 —— 先找画面里的边缘，再识别物体形状，最后理解物体间的空间关系==，最后把画面转换成 256 维的 “视觉特征”（相当于 AI 对画面的 “理解笔记”）。

第二部分，数值特征处理器：这是处理 “数据原料” 的模块，用两层全连接网络，把 85 个数据转换成 256 维的 “数值特征”（相当于 AI 对局势的 “数据总结”）。

第三部分，双流特征融合：把 “视觉特征（256 维）” 和 “数值特征（256 维）” 拼在一起，再经过两层网络处理，最终输出 Q 值——AI 选评分最高的动作，就能做出决策了。

简单说，这个架构就是 “把画面和数据都理解透，再融合起来做最优选择”，是 AI 能聪明决策的关键。

第三部分：分层决策架构

（切换到第一张 “底层动作模型” PPT）

现在我们来看**底层动作模型**的基础设定，这是 AI “做动作” 的底层规则：

首先是**动作空间定义**：AI 的 “操作库” 里有 9 个基础动作，比如上下左右移动、朝四个方向射击，还有一个 “等待” 动作 —— 相当于给 AI 配了一套 “基础操作按键”，所有复杂动作都是从这 9 个里组合出来的。

然后是**DQN 智能体初始化**：这里用了 Double DQN 的架构，简单说就是给 AI 配了 “两个大脑” —— 主网络负责实时决策，目标网络负责稳定判断；还准备了一个能存 10 万条经验的 “经验池”，让 AI 能反复学习过去的操作；另外有个 ε- 贪婪策略。

（切换到第二张 “底层动作模型” PPT）

接着看底层动作模型的**核心机制**：

先讲**动作选择机制（ε- 贪婪策略）**：训练时，AI 会 “偶尔随机选动作”（比如 ε=1.0 时全随机，慢慢降到 0.01），这是为了探索更多可能性；实战时就直接选 “得分最高的动作”，保证打得好 —— 相当于 “训练时多尝试，打仗时稳准狠”。

再看**目标网络更新策略**：目标网络不会频繁变，而是用 “软更新” 慢慢跟着主网络学（tau=0.001），这样 AI 的决策不会忽左忽右，能稳定提升 —— 就像给 AI 配了个 “稳重的教练”，主网络是 “灵活的选手”，教练慢慢学选手的新招，保证策略稳定。

（切换到 “高层选项模型” PPT）

最后是**高层选项模型**，这是 AI 的 “战术指挥系统”：

第一部分**战略选项定义**：AI 有 3 个大战术 —— 进攻、防守、绕后，这是指导底层动作的 “大方向”，比如选 “进攻” 战术，底层就会优先选 “前进 + 射击” 的动作；选 “防守” 就会优先 “护基地 + 躲子弹”。

第二部分**高层模型训练特点**：训练时，AI 选好战术（比如进攻）后，会用 “启发式函数” 生成对应的底层动作序列（比如先右移、再上射），把 “战略意图” 翻译成 “具体操作”—— 相当于 AI 先定 “打什么仗”，再想 “怎么打”，实现从 “战术” 到 “动作” 的落地

第四部分：TankAIAgent智能体实现

（切换到核心决策流程 1）

先看**核心决策流程**的前四步：

- **状态提取**：就是把游戏画面和数据，转成 AI 能懂的 “画面特征 + 数值特征”；
- **选战术**：AI 先定大方向（进攻 / 防守 / 绕后）；
- **选动作**：底层模型根据战术，选最优的基础动作；
- **安全检查**：紧急情况优先保基地、躲子弹 —— 比如敌人瞄准基地了，立刻切换安全动作。

（切换到核心决策流程 2）

后面三步是 “补漏 + 落地”：

- **动作微调**：比如防守时优先回基地；
- **防卡死**：AI 卡 bug 当连续等待动作超过30步时，就随机动一下；
- **转操作**：把 AI 的决策，变成游戏能识别的 “上下左右 / 射击” 指令。

（切换到关键安全检测算法）

最后是**安全算法**，这是 AI 的 “保命技能”：

- 基地瞄准检测：敌人瞄准基地就立刻防御；
- 子弹威胁检测：子弹快打到自己就躲； 这俩功能是让 AI “活下来” 的关键，能减少失误、稳定发挥。

简单说，这个智能体就是 “从‘看游戏’到‘做动作’的全流程”，还加了安全和防 bug 的保障，让 AI 能稳定打游戏

第五部分：训练系统设计

第一张 PPT：训练环境包装（对应 `ai_training_env.py`）

这是给 AI 搭的 “训练场地”，基本状态的定义，代码里的 `TrainingEnvironment` 类就是干这个的：

- **状态 / 动作定义**：代码里明确了 “AI 能看到什么（画面 + 85 个数据）、能做什么（9 个动作）”，和 PPT 里的状态维度、动作空间对应；
- **奖励函数**：PPT 里说的 “杀敌人 + 100、死了 - 200” 这些规则，代码里 `_calculate_reward` 函数全实现了 ——AI 做对了就加分、做错了就扣分，靠这个引导它学 “正确操作”。
- **step 方法**：AI 每做一个动作，这里就执行动作、算奖励、给新状态，是训练的 “一步流程”。

第二张 PPT：训练流程（对应 `ai_train.py`）

这是让 AI “在场地里练起来” 的过程，代码里 `train_dqn` 函数就是流程：

- **初始化**：先把训练环境、AI 智能体、经验池（存 AI 操作经验的地方）都准备好；
- **主循环**：AI 反复玩游戏（episode 是 “局数”），每一局里：
  1. 用 ε- 贪婪策略选动作（先乱试、后选最优）；
  2. 执行动作、往经验池存经验、算奖励；
  3. 经验够了就从池子里抽一批学（update 模型）；
  4. 定期更新目标网络、衰减 ε、存模型和数据；
- 最后保存训练好的模型和统计结果（比如每局得分）。

简单说：`ai_training_env.py` 是 “训练场地 + 打分规则”，`ai_train.py` 是 “让 AI 在场地里反复练、越练越好” 的过程，代码就是把 PPT 里的规则落地成可执行的流程。

第六部分：系统性能与评估

6.1 模型评估指标

第七部分：技术总结与创新点

先讲**核心技术特点**，这是咱们 AI 系统的 “核心优势”：

- **双模态特征融合**：简单说就是 AI 同时 “看画面 + 读数据”—— 用 CNN 处理游戏画面（比如敌人在哪），用全连接网络处理游戏数据（比如敌人血量），把这俩信息融合起来，让 AI 对游戏的认知又全又准，不会只看画面忽略关键数据，也不会只看数据搞不清场景。
- **分层决策机制**：AI 是 “先定战术，再做动作”—— 高层先选 “进攻 / 防守 / 绕后” 的大策略，底层再选 “上下左右 / 射击” 的具体动作，这样既保证 AI 有 “大局观”，又能把战术落地成具体操

作，比 “只凭本能乱打” 效率高得多。

- **安全强化学习**：给 AI 加了 “保命技能包”—— 比如优先保基地、躲子弹，结合奖励机制（比如保基地加分、被打扣分），让 AI 在 “赢游戏” 的同时先 “活下来”，避免一上来就被秒，保证系统能稳定运行。

（切换到 “工程实现创新” PPT）

再看**工程实现创新**，这是让系统 “好用、能落地” 的关键：

- **模块化架构**：把系统拆成 7 个核心文件（比如状态提取、模型训练各是一个文件），每个文件职责明确、接口统一，不管是改功能还是加新模块都很方便，开发和维护成本低。
- **训练优化**：用了标准化接口 + 多目标奖励 + 目标网络这些技术，让 AI“学得快、学得好”—— 比如多目标奖励既鼓励杀敌又鼓励生存，目标网络让训练更稳定，不用反复调参数试错。
- **部署友好**：把 AI 做的很 “轻”—— 模型大小不到 10MB，反应时间不到 10 毫秒，还能实时测试，不用高配设备就能跑，实际用起来很方便。

总结

我们的坦克大战AI系统成功实现了：

1. **完整的深度强化学习流程**：从状态感知到决策执行的全链路
2. **创新的分层架构**：解决复杂决策问题的有效途径
3. **工业级的可靠性**：多层安全机制确保系统稳定性
4. **优秀的性能表现**：在实时游戏中达到人类高水平

这套系统不仅是一个功能完整的游戏AI，更是深度强化学习在实际应用中的一次成功实践，为相关领域的研究和开发提供了宝贵的技术参考。

我的汇报到此结束，谢谢大家！